**TECHNICAL REPORT**

**Rescue Mission Order - Traveling Salesperson Variant**

**AI Project: Comparative Study of Search Algorithms vs. Genetic Algorithms**

---

**Team Information**

- **Project Title**: Rescue Mission Order (TSP Variant)
- **Team Size**: **Ibrahim Hashem – Ahmed Salah – Amany Mahmoud**
- **Programming Language**: Python 3
- **Libraries Used**: NumPy, Matplotlib, Random, Collections (deque)

---

## 1. ALGORITHM DESIGN

### 1.1 Problem Statement

A robot must rescue 5 victims (V1-V5) located in a complex grid environment with walls. The objective is to find the optimal **order** to visit all victims that minimizes the total travel distance.

### 1.2 Chromosome Representation

**Representation**: Each chromosome is a permutation of the integers [1, 2, 3, 4, 5].

**Example Chromosomes**:

- [3, 1, 5, 2, 4] means: Visit V3 first → V1 second → V5 third → V2 fourth → V4 fifth
- [4, 1, 5, 3, 2] means: Visit V4 first → V1 second → V5 third → V3 fourth → V2 fifth

**Why This Representation**:

- Each victim must be visited exactly once (permutation property)
- The order matters (different orders = different total distances)
- Simple and intuitive mapping to the problem domain
- No invalid chromosomes possible (all permutations are valid solutions)

### 1.3 Fitness Function - Mathematical Explanation

**CRITICAL: This is the core evaluation metric**

**Mathematical Formula:**

Fitness(chromosome) = Σ Distance(current_location → next_victim)

**Detailed Calculation Steps:**

For a chromosome C = [$c_1$, $c_2$, $c_3$, $c_4$, $c_5$]:

Fitness(C) = D(R → V_$c_1$) + D(V_$c_1$ → V_$c_2$) + D(V_$c_2$ → V_$c_3$) + D(V_$c_3$ → V_$c_4$) + D(V_$c_4$ → V_$c_5$)

Where:

- R = Robot starting position (index 0 in distance matrix)
- V_$c_i$ = Victim at position $c_i$ (index $c_i$ in distance matrix)
- D(A → B) = Distance from location A to location B (retrieved from distance_matrix)

**Example Calculation:**

**Given Chromosome**: [4, 1, 5, 3, 2]

**Distance Matrix** (from BFS results):

| | R | V1 | V2 | V3 | V4 | V5 |
|---|---|---|---|---|---|---|
| R | 0 | 18 | 12 | 26 | 10 | 30 |
| V1 | 18 | 0 | 26 | 28 | 14 | 14 |
| V2 | 12 | 26 | 0 | 14 | 20 | 30 |
| V3 | 26 | 28 | 14 | 0 | 34 | 16 |
| V4 | 10 | 14 | 20 | 34 | 0 | 26 |
| V5 | 30 | 14 | 30 | 16 | 26 | 0 |

**Step-by-Step Calculation**:

1. **Robot → V4**: distance_matrix[0][4] = 10 steps
2. **V4 → V1**: distance_matrix[4][1] = 14 steps
3. **V1 → V5**: distance_matrix[1][5] = 14 steps
4. **V5 → V3**: distance_matrix[5][3] = 16 steps
5. **V3 → V2**: distance_matrix[3][2] = 14 steps

**Total Fitness** = 10 + 14 + 14 + 16 + 14 = **68 steps**

**Key Properties:**

- **Lower fitness = Better solution** (minimization problem)
- Fitness is always a positive real number
- Uses actual pathfinding distances (BFS), not Euclidean distance
- Considers walls and obstacles in the environment

---

## 2. BFS PATHFINDING ALGORITHM

### 2.1 Purpose

Breadth-First Search (BFS) is used to calculate the **actual shortest path distance** between any two points in the grid, considering walls as obstacles.

### 2.2 Why BFS is Required

- **Straight-line (Euclidean) distance** would be incorrect because walls block direct paths
- BFS guarantees finding the **shortest path** in an unweighted grid
- Returns both distance and the actual path (for visualization)

### 2.3 Distance Matrix Construction

We build a 6×6 distance matrix where:

- **Index 0** = Robot starting position
- **Index 1-5** = Victims V1 through V5

The matrix is symmetric: D(A → B) = D(B → A)

### 2.4 Computational Complexity

- BFS runs **36 times** (6 locations × 6 locations)
- Each BFS: O(rows × cols) = O(400) for a 20×20 grid
- Total: O(36 × 400) = O(14,400) operations
- **Very fast**: Completes in < 0.1 seconds

---

## 3. GENETIC ALGORITHM PARAMETERS

### 3.1 Core Parameters

| Parameter | Value | Justification |
|---|---|---|
| **Population Size** | 100 | Large enough for diversity, small enough for speed |
| **Generations** | 500 | Sufficient for convergence on this problem size |
| **Mutation Rate** | 0.15 (15%) | Balanced exploration vs exploitation |
| **Crossover Rate** | 0.8 (80%) | High rate encourages recombination |
| **Elite Size** | 10 | Top 10% preserved to maintain good solutions |

### 3.2 Selection Method: Tournament Selection

- **Tournament Size**: 5 individuals
- **Process**: Randomly select 5 chromosomes, pick the best (lowest fitness)
- **Advantage**: Maintains selection pressure while preserving diversity

### 3.3 Crossover Method: Ordered Crossover (OX)

**Why Ordered Crossover?**

- Standard crossover would create **invalid chromosomes** (duplicate victims)
- OX preserves the permutation property
- Combines parental traits while maintaining validity

**How It Works**:

1. Select two random crossover points
2. Copy substring from Parent1 to Child
3. Fill remaining positions with genes from Parent2 in their original order

**Example**:

Parent1: [3, 1, | 5, 2 |, 4]

Parent2: [4, 5, | 2, 3 |, 1]

_____↓↓↓↓_____

Child: [4, 1, | 5, 2 |, 3]

### 3.4 Mutation Method: Swap Mutation

- **Process**: Randomly swap two positions in the chromosome
- **Probability**: 15% per chromosome
- **Purpose**: Introduce diversity and escape local optima

**Example**:

Before: [4, 1, 5, 3, 2]

↓ ↓

After: [4, 3, 5, 1, 2] (swapped positions 1 and 3)

### 3.5 Elitism

- The **top 10 chromosomes** (lowest fitness) are directly copied to the next generation
- Ensures best solutions are never lost
- Accelerates convergence

---

## 4. RESULTS AND COMPARISON

### 4.1 Distance Matrix (BFS Output)

Distance Matrix (in steps):

| | Robot | V1 | V2 | V3 | V4 | V5 |
|---|---|---|---|---|---|---|
| Robot | 0 | 18 | 12 | 26 | 10 | 30 |
| V1 | 18 | 0 | 26 | 28 | 14 | 14 |
| V2 | 12 | 26 | 0 | 14 | 20 | 30 |
| V3 | 26 | 28 | 14 | 0 | 34 | 16 |
| V4 | 10 | 14 | 20 | 34 | 0 | 26 |
| V5 | 30 | 14 | 30 | 16 | 26 | 0 |

### 4.2 Solution Comparison

| Method | Visit Order | Total Distance | Time |
|---|---|---|---|
| **Greedy (Baseline)** | [4, 1, 5, 3, 2] | 68 steps | < 0.01s |
| **Genetic Algorithm** | [4, 1, 5, 3, 2] | 68 steps | 0.42s |
| **Improvement** | - | 0 steps | - |

**4.3 Analysis**

**Why Same Result?** In this particular problem instance:

- The greedy algorithm (always choose nearest unvisited victim) found the **optimal solution**
- The GA confirmed this solution through population-based search
- Both methods converged to the same global optimum

**When GA Outperforms Greedy:**

- More complex environments with more victims (10+)
- When local greedy choices lead to poor global solutions
- Problems where the optimal solution requires "backtracking"

**4.4 GA Convergence Behavior**

- **Generation 0**: Best fitness = 82 steps (random initialization)
- **Generation 50**: Best fitness = 68 steps (optimal found)
- **Generation 50-499**: Stable at 68 steps (optimal maintained)
- **Average fitness**: Gradually improved from 108 → 71 steps

---

**5. VISUALIZATION OUTPUTS**

The program generates a comprehensive visualization with two plots:

**5.1 Left Plot: Grid Environment with Optimal Path**

- **Black cells**: Walls (obstacles)
- **Green square (R)**: Robot starting position
- **Colored circles**: Five victims (V1-V5)

- **Yellow path**: Optimal rescue route
- **Arrows**: Direction of travel

---

## 6. CODE STRUCTURE

### 6.1 Main Components

1. **Environment Setup** (create_complex_environment)
    - Creates 20×20 grid with strategic wall placement
    - Places robot and 5 victims in challenging positions
2. **BFS Pathfinding** (bfs_shortest_path)
    - Implements queue-based BFS
    - Returns distance and actual path
    - Handles walls and boundaries
3. **Distance Matrix Builder** (build_distance_matrix)
    - Runs BFS for all location pairs
    - Stores results in 6×6 matrix
4. **Genetic Algorithm Class** (GeneticAlgorithm)
    - Fitness calculation
    - Population initialization
    - Selection, crossover, mutation
    - Evolution loop with statistics tracking
5. **Visualization** (visualize_solution)
    - Matplotlib-based grid rendering
    - Path drawing with directional arrows

### 6.2 Code Quality Features

- **Modular design**: Clear separation of concerns
- **Comprehensive comments**: Every function documented

- **Error handling**: Boundary checks and validation
- **Readable variable names**: Self-documenting code

---

**7. KEY INSIGHTS AND LEARNING**

**7.1 Chromosome Design Insight**

- **Permutation representation** is perfect for ordering problems
- Avoids invalid solutions by design
- Simplifies crossover and mutation operators

**7.2 Fitness Function Insight**

- Must accurately model the optimization objective
- Using BFS distances (not Euclidean) is crucial for realistic pathfinding
- Lower fitness = better solution (minimization)

**7.3 GA Parameter Tuning**

- Population size affects diversity vs computation time
- Mutation rate balances exploration vs exploitation
- Elitism prevents losing good solutions
- Crossover method must preserve chromosome validity

**7.4 Search Algorithm Comparison**

- **BFS Strength**: Guarantees shortest path between two points
- **GA Strength**: Finds good ordering/sequencing among many locations
- **Complementary**: BFS feeds accurate data to GA for optimization

---

**8. MANUAL CALCULATION EXAMPLE (FOR ORAL DEFENSE)**

**Question**: Calculate the fitness of chromosome [2, 4, 3, 5, 1]

**Solution**:

Using the distance matrix:

1. Robot → V2: distance_matrix[0][2] = 12 steps
2. V2 → V4: distance_matrix[2][4] = 20 steps
3. V4 → V3: distance_matrix[4][3] = 34 steps
4. V3 → V5: distance_matrix[3][5] = 16 steps
5. V5 → V1: distance_matrix[5][1] = 14 steps

**Total Fitness = 12 + 20 + 34 + 16 + 14 = 96 steps**

This is **worse** than the optimal solution (68 steps), so this chromosome would have lower selection probability.

---

## 9. CONCLUSION

### 9.1 Project Success

✓ Successfully implemented BFS for accurate distance calculation
✓ Built a working Genetic Algorithm for TSP optimization
✓ Generated clear visualizations of the solution
✓ Demonstrated understanding of both algorithms

### 9.2 Comparative Analysis

- **BFS**: Fast, deterministic, finds shortest paths between pairs
- **GA**: Heuristic, stochastic, optimizes global ordering
- **Combined Approach**: Leverages strengths of both methods

### 9.3 Real-World Applications

- Emergency response routing
- Delivery truck optimization
- Drone path planning
- Warehouse robot scheduling

---

## 10. TEAM CONTRIBUTIONS

**Member 1: Algorithm Implementation**

- Implemented BFS pathfinding algorithm
- Built distance matrix generation
- Debugged path calculation logic

**Member 2: Genetic Algorithm Development**

- Designed chromosome representation
- Implemented fitness function
- Coded selection, crossover, and mutation operators
- Tuned GA parameters

**Member 3: Visualization and Testing**

- Created matplotlib visualization code
- Tested various environment configurations
- Generated grid environment visualizations
- Wrote technical report documentation

---

**APPENDIX A: DISTANCE CALCULATION DETAILS**

**Full Distance Matrix with Interpretations:**

**Robot to All Victims**:

- R → V1: 18 steps (far right, requires navigating walls)
- R → V2: 12 steps (left side, relatively close)
- R → V3: 26 steps (bottom center, far distance)
- R → V4: 10 steps (top center, shortest from robot)
- R → V5: 30 steps (bottom right corner, longest from robot)

**Victim-to-Victim Key Distances**:

- V1 ↔ V5: 14 steps (both on right side)
- V2 ↔ V3: 14 steps (left-bottom connection)
- V3 ↔ V5: 16 steps (bottom region)

- V4 ↔ V1: 14 steps (top to right)

---

**APPENDIX B: RUNNING THE CODE**

**Requirements:**

pip install numpy matplotlib

**Execution:**

python main.py

**Expected Output:**

1. Console output with step-by-step progress
2. Distance matrix printout
3. Generation-by-generation GA statistics
4. Final comparison results
5. Visualization saved as rescue_mission_solution.png